

Codex 初探

-运行机制与最佳实践

分享人：王谊杰
时间：
2026.05.15

1. 智能体循环：运行机制与核心原理
2. 安全边界：沙箱与审批
3. 上下文工程：让codex知道怎么干活
4. 能力扩展：MCP
5. 流程复用：Skills
6. 并行协作：Subagent
7. 最佳实践与落地建议
8. ChatGPT 会员订阅最佳实践



1. 智能体循环：运行机制与核心原理

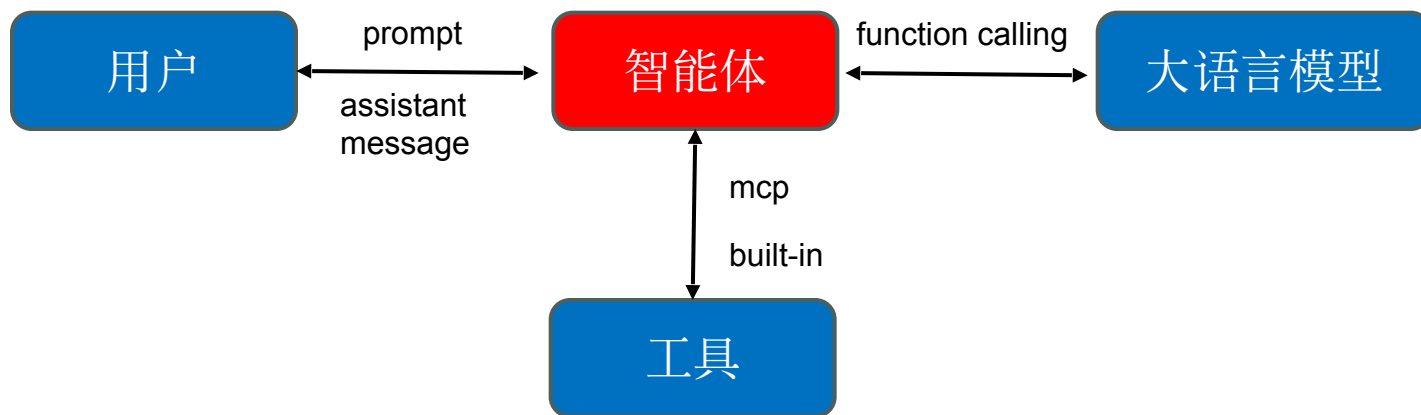
1. 智能体循环：运行机制与核心原理



邦盛科技
Bangsun Technology

>什么是智能体？

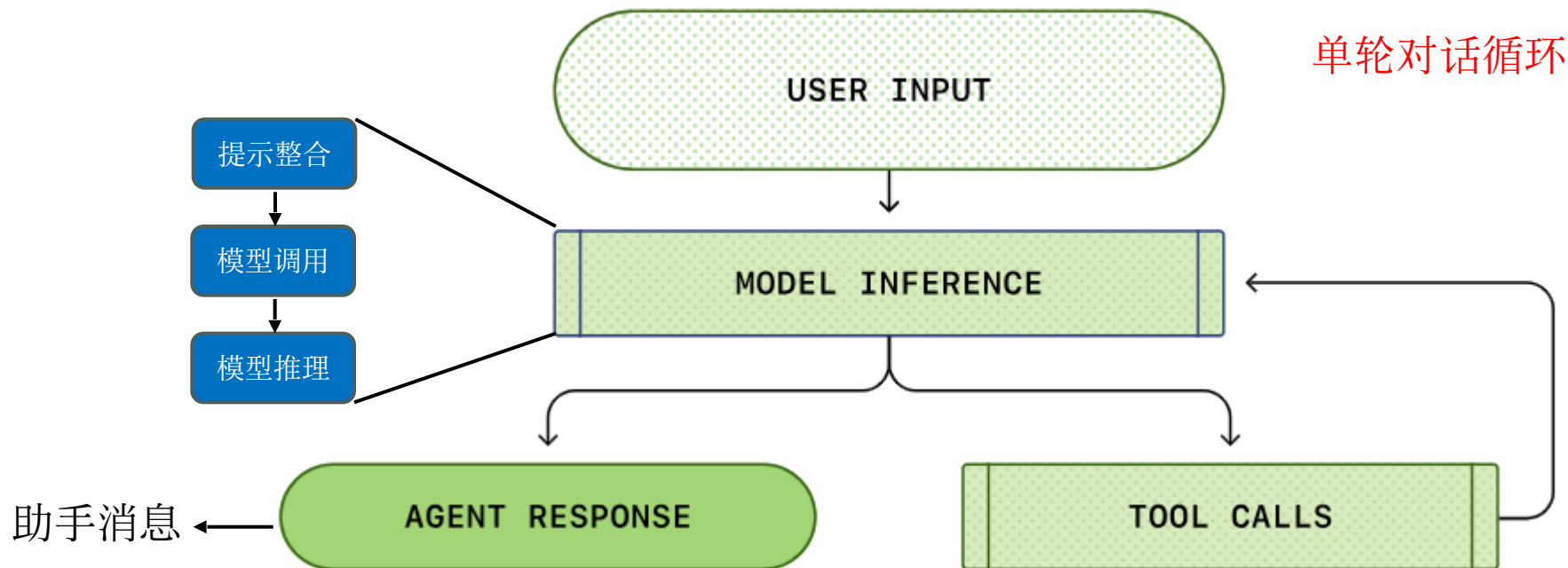
类似于编排器，负责协调用户、模型以及模型为执行任务所调用的工具三者之间交互。



1. 智能体循环：运行机制与核心原理

>智能体是如何协调用户、模型以及工具来完成用户下达的任务的？

①首先，用户输入信息先到达智能体，②智能体整合后，将信息发给大语言模型，大语言模型进行推理，并将推理结果返回给智能体。③智能体根据推理结果决定调用工具还是将推理结果返回给用户结束该轮对话。④如果调用工具，则将工具执行结果返回给大模型，再次执行模型推理，直至模型不再调用工具，转而生成助手消息。



1. 智能体循环：运行机制与核心原理

> 多轮对话呢？智能体是如何运行

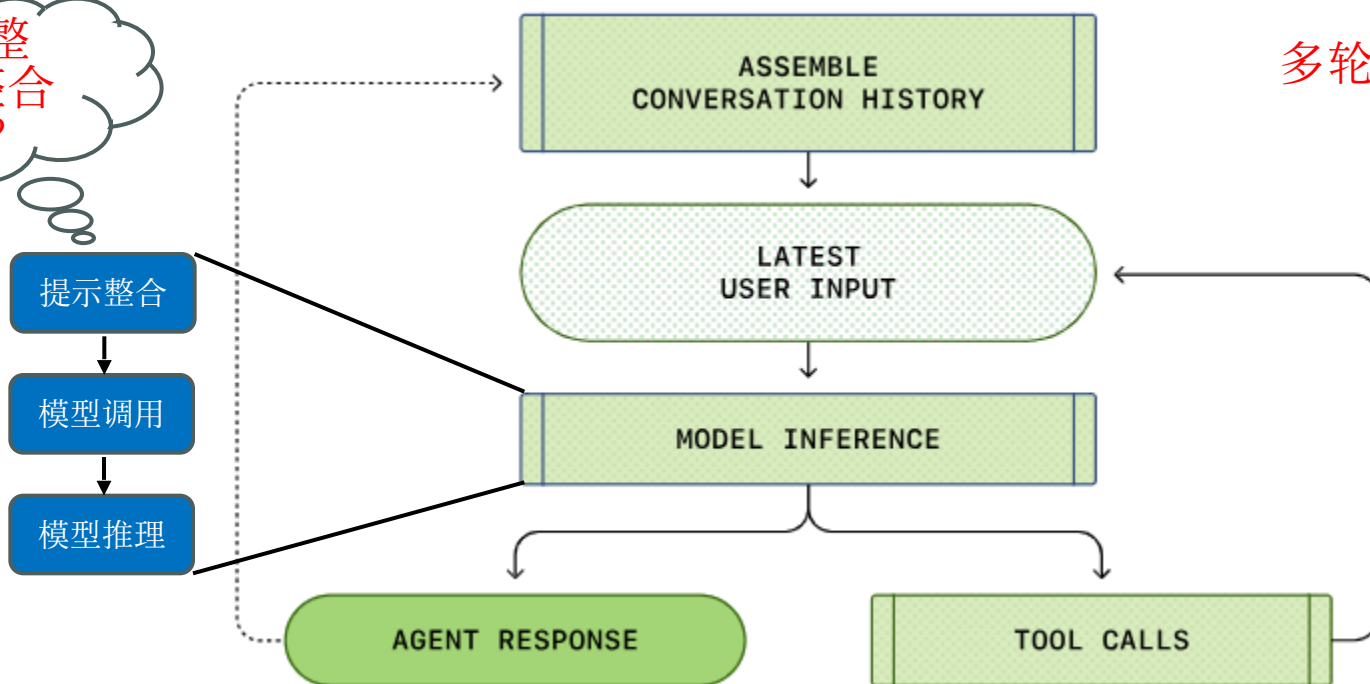
的？
当进行第二轮对话，智能体会将之前所有对话轮次的内容与新的用户输入整合作为大语言模型的提示。

智能体在单轮对话中就可能发起上百次工具调用，而每个模型的上下文窗口是有限制的，随着对话轮次增多，上下文窗口耗尽不可避免。

输入+输出

多轮对话循环

如何整合？整合什么？



1. 智能体循环：运行机制与核心原理

> 智能体如何整合提示？发送给大语言模型的提示中有哪些信息？

Codex 通过向 Responses API 发送 HTTP 请求来执行模型推理。在发送 HTTP 请求之前会进行提示信息的构建。这不仅仅包括用户的输入，还包括指令、工具、权限等信息。

智能体构建的初始提示
(API 层)

"instructions": "You are Codex, an AI coding agent. You should..."

```
"tools": [
  {
    "type": "function",
    "name": "shell",
    "description": "Run a shell command on the local machine",
    "parameters": {
      "type": "object",
      "properties": {
        "command": {
          "type": "array",
          "items": { "type": "string" },
          "description": "Command to execute"
        },
        "workdir": {
          "type": "string",
          "description": "Working directory"
        }
      },
      "required": ["command"]
    }
  }
]
```

来源 config.toml 或
codex 内置文件

```
{
  "instructions": "...",
  "tools": [...],
  "input": [...]
}
```

```
"input": [
  {
    "type": "message",
    "role": "developer",
    "content": [
      {
        "type": "input_text",
        "text": "<permissions instructions>\n- You can read files\n- You can write to workspace\n- Ask before dangerous operations\n</permissions instructions>"
      }
    ]
  }
]
```

1. 智能体循环：运行机制与核心原理

>**codex**构建的初始提示就是模型最终看到的信息吗？

Responses API 服务端接受到codex构建的初始提示后会对其进行进一步地处理。将API提示分为system > developer > user > assistant四种提示。

优先级由高到低

Role: ● System ● Developer ○ User

模型看见的初始提示

服务端控制，
我们无法改变

SYSTEM: You are ChatGPT...

DEVELOPER: tools (展开成能力描述)

DEVELOPER: instructions

DEVELOPER: permissions

DEVELOPER: developer_instructions

USER: AGENTS.md

USER: environment_context

USER: 用户输入

Beginning of prompt

Order is controlled
by the server

Items from "input"
to the Responses API

Order is user controlled

"You are ChatGPT..."

Tools: derived from "tools" in the API call

Instructions: from "instructions" in the API call

<permissions instructions>

developer_instructions from config.toml

AGENTS.md via get_user_instructions()

<environment_context>

"Add an architecture diagram to the README.md"

Derived by Codex in
build_initial_
context()

Provided by user

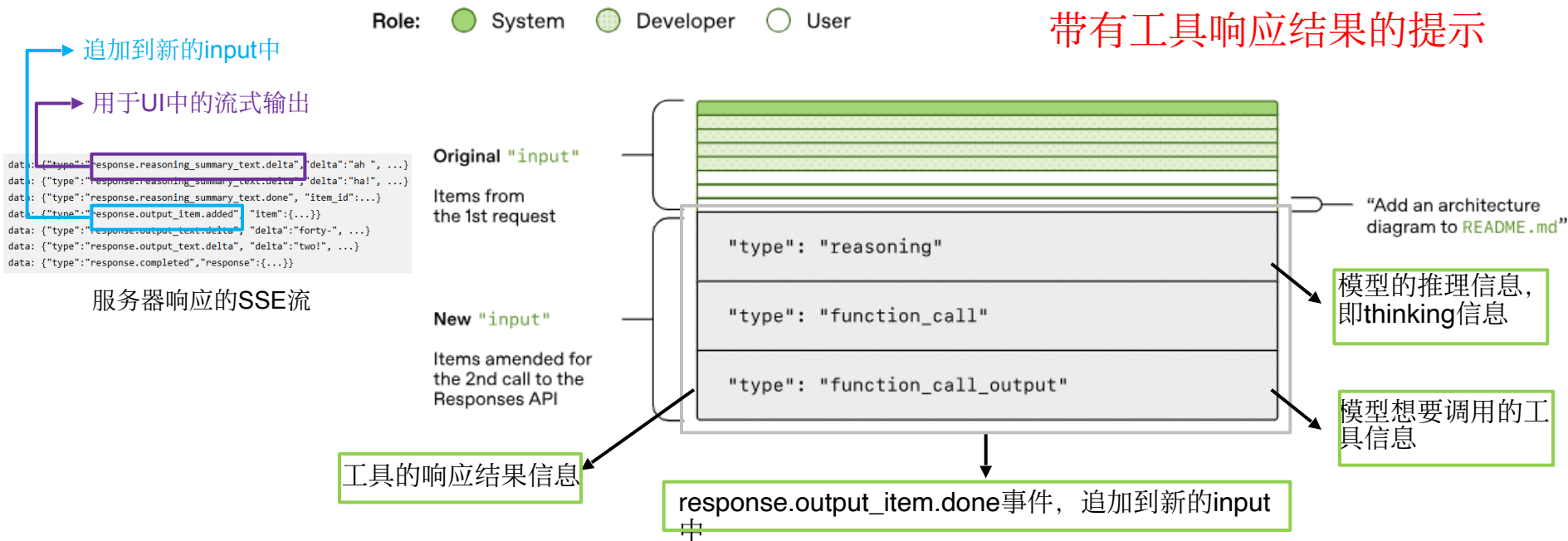
1. 智能体循环：运行机制与核心原理



> 服务器是如何响应**codex**的？**Codex**如何知道该调用工具还是停止对话？

服务器会回复一个服务器发送事件 (SSE) 流。每个事件的 **data** 是一个 JSON 负载，其“**type**”字段以“**response**”开头。

codex针对不同的**type**值采取不同的措施，包括调用工具、发送助手消息结束本轮会话等。



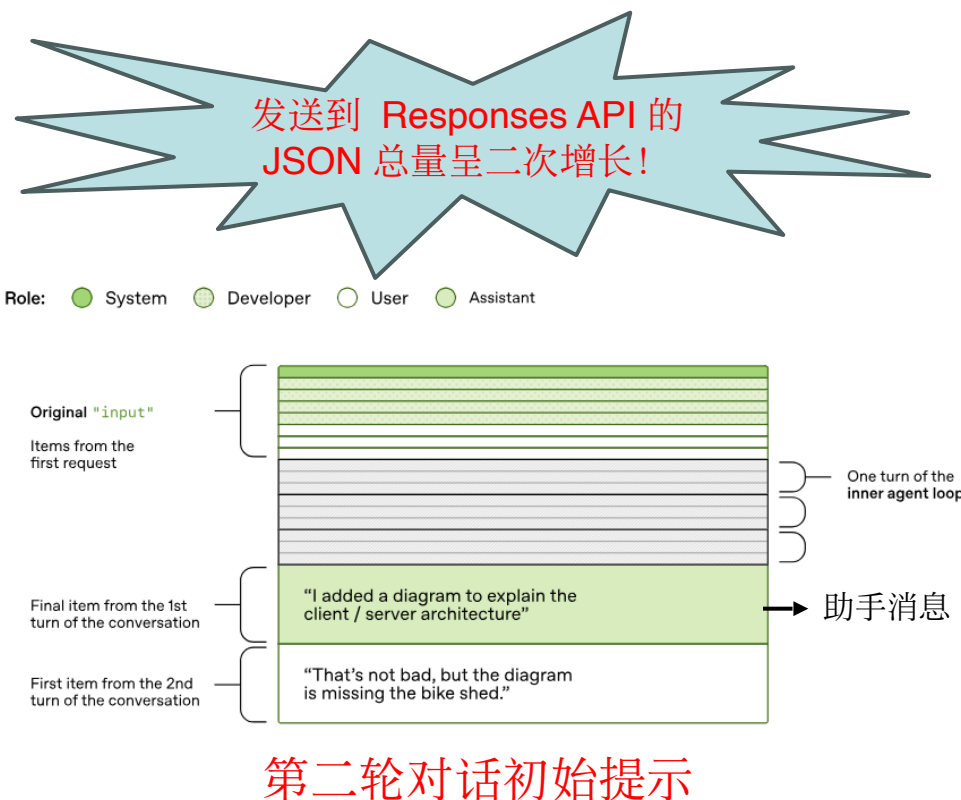
1. 智能体循环：运行机制与核心原理

>好，我明白了，你刚刚说的只是第一轮对话的运行机制，第二轮会话是如何运行的？

当codex接受到一个助手消息事件时，codex将助手消息输入到UI，向用户展示，第一轮对话也就结束了。

如果用户做出回应，则上一轮整段对话和用户的新消息都会追加到 Responses API 请求的 input 中，开始新一轮对话。

```
[
  /* ... all items from the last Responses API request ... */
  {
    "type": "message",
    "role": "assistant",
    "content": [
      {
        "type": "output_text",
        "text": "I added a diagram to explain the client/server architecture."
      }
    ]
  },
  {
    "type": "message",
    "role": "user",
    "content": [
      {
        "type": "input_text",
        "text": "That's not bad, but the diagram is missing the bike shed."
      }
    ]
  }
]
```

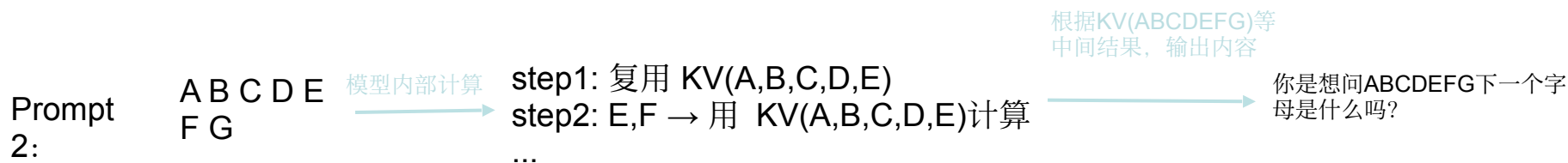
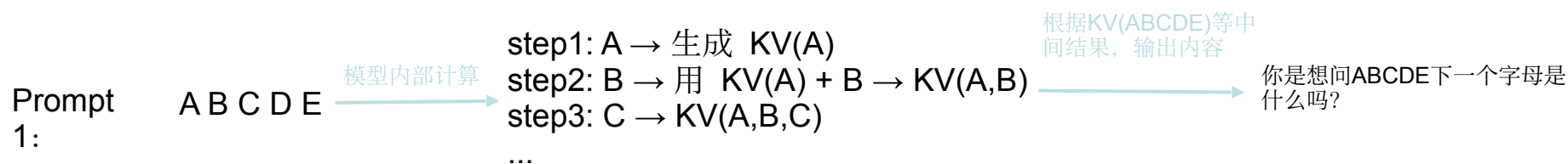


1. 智能体循环：运行机制与核心原理

>随着对话轮次的不断增加，我套餐内**token**的消耗岂不是也是二次增长？

针对该问题，为了提高计算资源的利用率，以及减少用户**token**的损耗，**codex**通过提示缓存来解决这个问题。

模型在第一次推理后会保存其经过模型计算的中间结果（如**self-attention**计算的k、v），当第二次推理发生时，如果提示转换后的**token**的前缀与第一次推理的提示**token**完全匹配，则第二推理的前缀部分使用保存的中间数值，不再重复计算，只计算没有匹配到的部分，这就是**codex**提示缓存的原理。



1. 智能体循环：运行机制与核心原理

> 哪些操作可能导致 **codex** 发生“缓存未命中”？

提示缓存就是复用已经算过的前缀状态，减少模型计算量，提高计算资源使用率。如果新 **prompt** 的开头和旧 **prompt** 完全一样，就可以复用之前的计算结果。其复用过程就叫缓存命中。

以下操作可能导致缓存未命中：

1. 在对话中途更改模型可用的 **tools**。
2. 更改作为 **Responses API** 请求目标的模型（实际上，这会更改原始提示中的第三项，因为该项包含模型特定的指令）。
3. 更改沙盒配置、审批模式或当前工作目录。

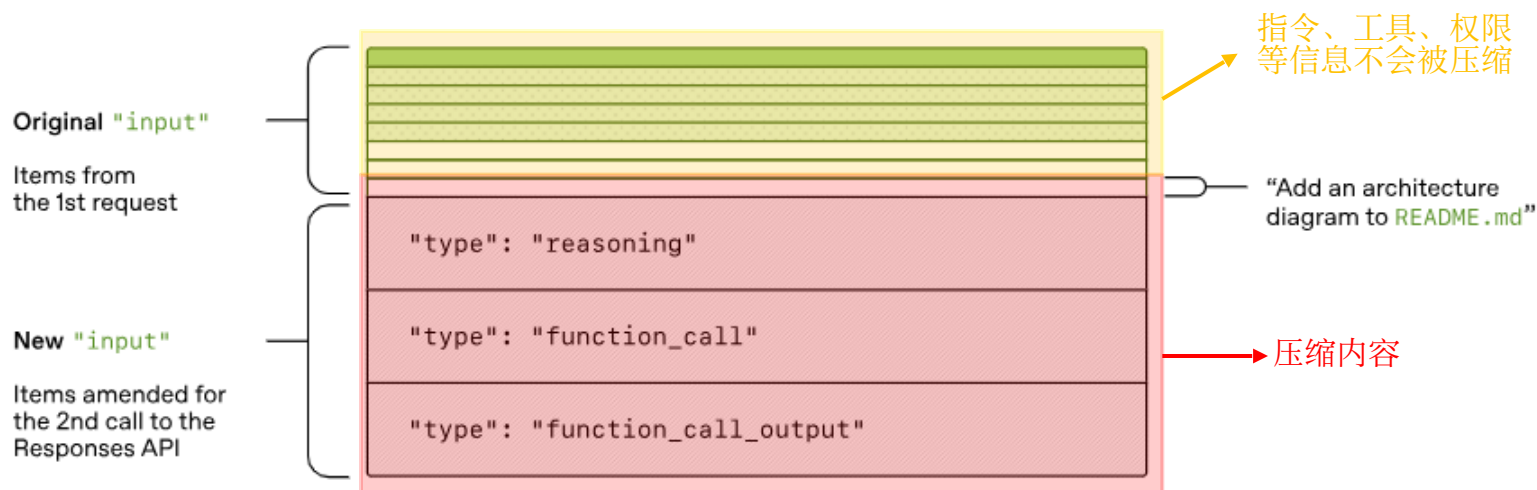
1. 智能体循环：运行机制与核心原理

>**compact** 操作会压缩哪些信息？**AGENT.md**会被压缩吗？

当上下文窗口超过模型的阈值时，**codex**会对提示进行自动压缩。当然你也可以手动触发该操作，提前压缩。

压缩只会发生在用户的对话历史，如：多轮 **user / assistant message**、工具调用记录（**function_call / output**）、**reasoning**（包括 **encrypted_content**）、长日志、代码输出等

Role: ● System ● Developer ○ User





2.安全边界：沙箱与审批

2.安全边界：沙箱与审批

>什么是沙箱？什么是审批？在**codex**中的作用是什么？

Codex CLI 的安全控制主要由两层组成：沙箱 `sandbox_mode` 和 审批 `approval_policy`。

沙箱决定 Codex “能做什么，执行边界”，常见的`sandbox_mode`：

- `read-only`：只读模式。Codex 可以查看文件，但不能直接修改文件或执行需要写入/副作用的命令。
- `workspace-write`：工作区可写。Codex 可以读取文件、修改当前 `workspace` 内文件，并运行常规本地命令。
- `danger-full-access`：完全访问。取消文件系统和网络边界，Codex 基本按当前用户权限执行。

跨域沙箱边界

审批决定 Codex “什么时候必须先问你”，常见的`approval_policy`：

- `untrusted`：不在可信集合内的命令需要审批。
- `on-request`：需要越过沙箱边界，例如访问网络、修改 `workspace` 外文件，以及危险的命令会发起请求审批。
- `never`：不会停止以请求审批。

2.安全边界：沙箱与审批

>其它与安全边界的配置信息还有哪些？

Codex CLI 常见的安全控制还有`approvals_reviewer`、`windows.sandbox`、`writable_roots`以及`network_access`。

approvals_reviewer：指定审批对象。**user**：审批请求会显示给用户；**auto_review**：符合条件的审批请求会发送给审核代理。可以通过`[auto_review].policy`对`auto_review`内容进行进一步的定制。

windows.sandbox：windows系统特有配置，配置windows沙箱的使用模式，“elevated”或“unelevated”，默认为“elevated”。“elevated”更为严格，“unelevated”限制更宽松。

writable_roots：在sandbox基础上扩展可修改的路径。

network_access：是否可访问网络资源，默认不可以。开启时默认访问openai缓存的网络资源，如果需要访问实时的网络数据，需配置`web_search = "live"`



3.上下文工程：让codex知道怎么干活

3.上下文工程：让codex知道怎么干活

>可配置的上下文有哪些？

Codex 可配置的上下文信息有很多，如config.toml、AGENTS.md和subagent定义.md文档等。

config.toml：配置codex的默认行为或全局信息。可配置默认模型、沙箱模式、审批策略、**MCP Server**、**skills**等信息。可在~/.codex和项目根目录下配置。项目根目录下的config.toml文件优先级大于~/.codex下的。

AGENTS.md：codex的用户级指令文件。配置codex需要遵守的规则，类似于Cloude Code中的CLOUDE.md文件。可在~/.codex、项目根目录、项目各目录下配置。离codex启动目录越近优先级越高。~/.codex/AGENTS.md文件适用于所有项目。

在codex中可以配置多份AGENTS.md文件，其加载规则为从当前工作目录开始向上级目录搜寻AGENTS.md文件，一直到项目根目录，将搜寻到的所有AGENTS.md文件都加载到提示中。

并且AGENTS.override.md文件将覆盖相同目录下的AGENTS.md文件内容。



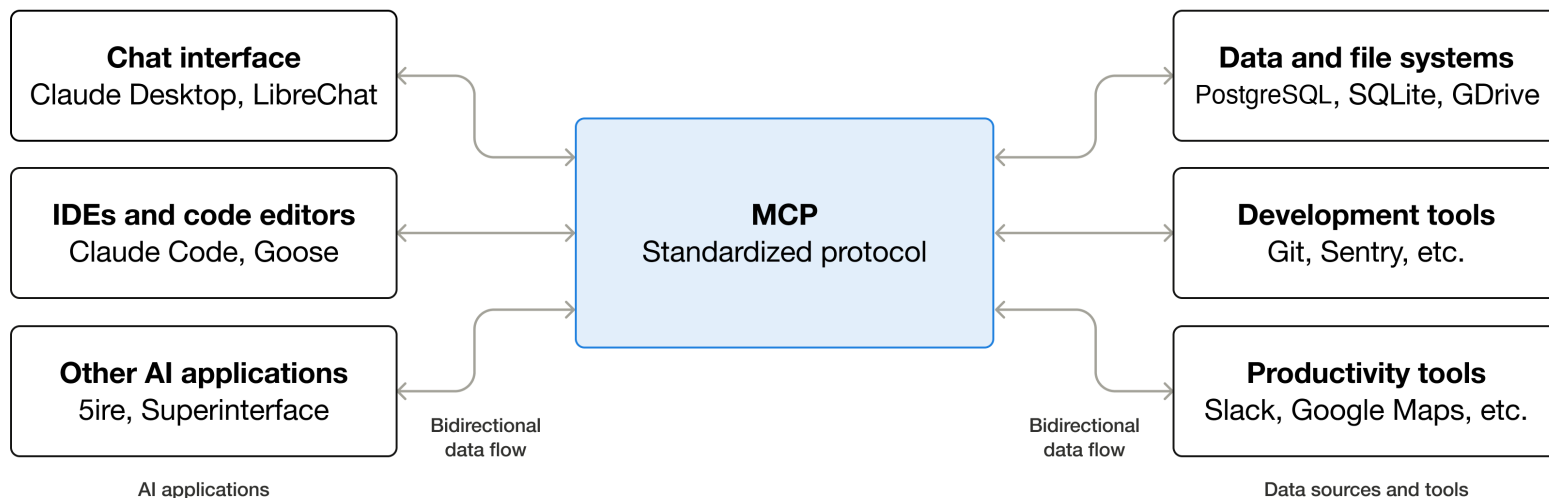
4.能力扩展：MCP

4.能力扩展： MCP

>什么是MCP?

MCP（Model Context Protocol，模型上下文协议）是一种用于将 AI 应用程序连接到外部系统的开源标准。

通过使用 MCP，像 Claude 或 ChatGPT 这样的 AI 应用可以连接到数据源（例如本地文件、数据库）、工具（例如搜索引擎、计算器）以及工作流（例如专用提示）——从而使它们能够访问关键信息并执行任务。



注：MCP是智能体与大语言模型直接交互的标准，function call是智能体与大语言模型之间交互的标准。

4.能力扩展： MCP

>codex中如何使用MCP?

Codex支持两种类型MCP服务

- **STDIO** 服务器：作为本地进程运行的服务器（通过命令启动）
- 可流式 **HTTP** 服务器（**Streamable HTTP servers**）：通过地址访问的服务器

MCP 的配置信息存储在 `config.toml` 中，codex有两种配置MCP服务器的方式：

1. 使用 CLI：运行 `codex mcp` 来添加和管理服务器编辑
2. `config.toml`：直接更新 `~/.codex/config.toml`（或项目级 `.codex/config.toml`）

CLI 配置指令：

```
codex mcp add <server-name> --env VAR1=VALUE1 --env VAR2=VALUE2 -- <stdio server-command>
```

4.能力扩展：MCP

>**codex**中**STDIO**可配置项?

STDIO 服务器:

- **command** (必需): 启动服务器的命令
- **args** (可选): 传递给服务器的参数
- **env** (可选): 为服务器设置的环境变量
- **env_vars** (可选): 允许并转发的环境变量
- **cwd** (可选): 服务器启动时的工作目录
- **experimental_environment** (可选): 当有远程执行环境时, 设置为 **remote** 以通过远程环境启动 **stdio** 服务器

env_vars 可以包含普通变量名或带来源的对象:

❏ TOML

```
env_vars = ["LOCAL_TOKEN", { name = "REMOTE_TOKEN", source = "remote" }]
```

4.能力扩展： MCP

>**codex**中可流式 **HTTP**可配置项?

可流式 **HTTP** 服务

• 器 url（必需）： 服务器地址

- bearer_token_env_var（可选）： 用于在 Authorization 中发送 Bearer token 的环境变量名
- http_headers（可选）： 静态 header 映射
- env_http_headers（可选）： header 名到环境变量名的映射（值从环境中读取）

其它配置

项:startup_timeout_sec（可选）： 服务器启动超时（秒）， 默认 10

- tool_timeout_sec（可选）： 工具运行超时（秒）， 默认 60
- enabled（可选）： 设为 false 可禁用服务器而不删除
- required（可选）： 设为 true 时， 如果服务器无法初始化则启动失败
- enabled_tools（可选）： 工具白名单
- disabled_tools（可选）： 工具黑名单（在 enabled_tools 之后应用）

4.能力扩展：MCP

>codex 中 MCP 实际使用

STDIO 服务器（以context7为例）：

例）：

1.CLI:

```
codex mcp add context7 -- npx -y @upstash/context7-mcp -- api-key YOUR_API_KEY
```

2. Config.toml:

```
[mcp_servers.context7]
command = "npx"
args = ["-y", "@upstash/context7-mcp", "--api-key", "YOUR_API_KEY"]
startup_timeout_sec = 40
```

使用：

```
> 帮我查一下 Spring Boot 3.2 的配置写法

• 我先按 Spring Boot 3.2 官方文档核对配置文件、类型安全配置和 profile 的写法，避免按旧版本经验回答。
• Called context7.resolve-library-id({"libraryName":"Spring Boot","query":"Spring Boot 3.2 externalized configuration application.yml @ConfigurationProperties profiles config import official documentation"})
  Available libraries:
  - Title: Spring Boot
  - Context7-compatible library ID: /spring-projects/spring-boot
  - Description: Spring Boot helps you to create Spring-powered, production-grade applications and services with absolute minimum fuss.
  - Code Snippets: 816
  - Source Reputation: High
  - Benchmark Score: 76.24
  - Versions: v2.5.5, v3.4.1, v2.7.18, v3.3.11, v3.5.3, v3.1.12, v4.0.0_m1, v4.0.0_m2, v4.0.0, v2.2.8.release, v3.5.9, v4.0.3
  - Description: Spring Boot simplifies the creation of stand-alone, production-grade Spring applications with embedded servers and provides extensive features for monitoring and management.
  - Code Snippets: 113724
  - Title: Spring Boot
  - Context7-compatible library ID: /websites/spring_io_spring-boot_3_5
  - Description: Spring Boot simplifies the creation of stand-alone, production-grade Spring applications by providing ...
```


4.能力扩展：MCP



邦盛科技
Bangsun Technology

>codex 中 MCP 实际使用

可流式 HTTP 服务器（以context7为例）：

1.CLI:

```
codex mcp add openaiDeveloperDocs --url https://developers.openai.com/mcp
```

2. Config.toml:

```
[mcp_servers.openaiDeveloperDocs]
url = "https://developers.openai.com/mcp"
```

配合用户指令文档（**AGENTS.md**）使用

使用：通用开发原则

- 在实现任何方法或功能前，必须先检查当前代码库中是否已存在可复用的接口或实现；若存在，必须优先复用并禁止重复造轮子，且复用时必须遵循本规范中的接口复用规则
- 如需使用 OpenAI API、ChatGPT 应用软件开发工具包、Codex 等 OpenAI 相关功能，必须始终调用 OpenAI 开发文档 MCP 服务器查询官方文档；无需用户额外提醒，不得仅凭记忆或经验直接回答或实现

› 深入解析 Codex 智能体循环

▲Falling back from WebSockets to HTTPS transport. timeout waiting for child process to exit

• 我会先查官方 OpenAI/Codex 文档，确保对“智能体循环”的说明基于当前文档，而不是只凭经验概括。

• Ran Get-Content -Raw 'C:\Users\bangsun\.codex\skills\.system\openai-docs\SKILL.md'
... +86 lines (ctrl + t to view transcript)

• 我正在使用 openai-docs 技能，因为问题涉及 Codex/OpenAI 产品机制；接下来会只取和 Codex、Agents 循环直接相关的官方材料。

• Called openaiDeveloperDocs.search_openai_docs({ "query": "Codex coding agent overview task environment tool use", "limit": 5 })
[{"hits": [{"url": "https://developers.openai.com/cookbook/examples/gpt-5/gpt-5_prompting_guide#system-prompt-and-parameter-tuning-verbosity-and-autonomous-behavior-while-giving-users-the-ability-to-configure-custom-instructions", "anchor": "system-prompt-and-parameter-tuning-verbosity-and-autonomous-behavior-while-giving-users-the-ability-to-configure-custom-instructions", "text": "The team initially found that during long horizon tasks, while still faithfully following user-provided instructions. The team initially found that summaries that, while technically relevant, disrupted the natural flow of the user; at the same time, the code outputted th single-letter variable names dominant. In search of a bett..."}]



5.流程复用: Skills

5.流程复用： Skills

>什么是Skills?

给智能体提供的“可复用工作流程模板”，为智能体扩展特定任务的能力。

Skills是可复用工作流的编写格式。**Codex**使用渐进式披露（**progressive disclosure**）来高效管理上下文：**Codex** 初始只会看到每个技能名称、描述和文件路径。只有当 **Codex** 决定使用某个技能时，才会加载完整的 **SKILL.md** 指令

Codex 会在上下文中包含可用技能的列表。为了避免挤占提示（**prompt**）中的其他内容，这个列表的大小被限制为模型上下文窗口的大约 **2%**，或者在上下文窗口未知时为 **8,000** 个字符。如果安装了很多技能，**Codex** 会首先缩短技能描述。在技能集合非常大的情况下，一些技能可能不会出现在初始列表中，并且 **Codex** 会显示警告。

一个技能是一个目录，其中包含一个 **SKILL.md** 文件以及可选的脚本和参考资料。**SKILL.md** 文件必须包含 **name** 和 **description**。

```
my-skill/  
  SKILL.md      (必需: 指令 + 元数据)  
  scripts/      (可选: 可执行代码)  
  references/    (可选: 文档)  
  assets/        (可选: 模板、资源)  
  agents/  
    openai.yaml (可选: 界面配置和依赖)
```

名称为my-skill的skill
内部结构

5.流程复用: Skills



>如何使用Skills?

创建技能:

1. **\$skill-creator**: 该创建器会询问技能的功能、触发条件, 以及是否仅包含指令或需要脚本。
2. 手动创建: 创建一个包含SKILL.md文件夹, 并且该md文件必须包含name和description。
3. **\$skill-installer**: 从远程仓库下载已创建好的skill。如果从第三方仓库下载则需要使用全路

调用技能:

- 显式调用: 在提示中直接包含技能。在 CLI/IDE 中运行 `/skills` 或输入 `$` 来提及一个技能。
- 隐式调用: 当任务匹配技能的 `description` 时, Codex 会自动选择该技能。

最佳实践:

1. 每个技能只做一件事
2. 优先使用指令而不是脚本 (除非需要确定性行为)
3. 编写清晰的步骤 (输入与输出)
4. 用实际提示测试技能描述是否能正确触发

强依赖于 **description**

结合MCP的**SKILL**编写:

注: 可以在**config.toml**中开启或关闭某个**skill**



6.并行协作：Subagent

6.并行协作： Subagent

>什么是**Subagent**?

子代理（**Subagent**）就是你让主智能体再生成多个子智能体，分别去做不同子任务，然后把结果汇总回来。可以在一定程度上加速任务处理，缓解上下文污染（context pollution）和上下文腐化（context rot）等问题。

子代理配置和使

用

- 1.子代理默认继承使用主代理的配置。

- 2.可以通过在`./codex/agent/`下建配置文件，自定义子代理行为

- 3.在`./codex/config.toml`中配置子代理的深度和最多允许存在的子代理数量

- 2.直接在提示中说明启动多个代理执行任务



7.最佳实践与落地建议

7.最佳实践与落地建议

困难任务先规划

如果任务复杂、模糊或难以描述，可以在开始编码前让 **Codex** 先制定计划。

使用 **Plan** 模式：对大多数用户来说，这是最简单有效的方式。**Plan** 模式允许 **Codex** 收集上下文、提出澄清问题并在实现前构建更好的计划。通过 `/plan` 或 `shift+tab` 切换该模式。

让 **Codex** 采访你：如果你对目标只有一个大致想法但难以描述，可以让 **Codex** 先提问。让它挑战你的假设，并在写代码之前将模糊想法转化为具体方案。

使用 **PLANS.md** 模板：对于更高级的工作流，可以配置 **Codex** 遵循 **PLANS.md** 或执行计划模板来处理长时间或多步骤任务。详见执行计划指南。

7.最佳实践与落地建议

PLAN模式实践

1.描述需求（**shift+tab**进入plan模式）

› 在SdModelExtController类中新增一个接口，该接口给后台组件提供服务，入参为模型id和版本，出参为ModelOutput中的paramKey和field

gpt-5.5 medium · Context 63% left · 5h 93% · weekly 83% · Fast off

Plan mode (shift+tab to cycle)

2.Codex进行反向提问，确认实现细节

已找到可复用方法：`ModelQueryManager#getModelOutputs(modelId, version, keyword)` 会按模型ID和版本读取 `outputs` 并解析为 `ModelOutput`，`version` 为空或 0 时还会按最新版本处理。现在需要明确新接口的对外契约，尤其是路径和返回结构，避免新增了后台组件无法对接的形态。

Question 1/2 (2 unanswered)

这个新增接口希望采用哪种入参与返回契约？

- | | |
|----------------------|---|
| › 1. GET查询参数（推荐） | 新增 <code>`GET /rs/model/ext/outputs/keys?modelId=...&version=...`</code> ，返回仅含 <code>`paramKey`</code> 和 <code>`field`</code> 的列表，适合后台组件读取。 |
| 2. POST JSON | 新增 <code>`POST /rs/model/ext/outputs/keys`</code> ，请求体包含 <code>`modelId`</code> 和 <code>`version`</code> ，便于后续扩展更多筛选条件。 |
| 3. 复用路径风格 | 新增 <code>`GET /rs/model/ext/{modelId}/{version}/outputs/keys`</code> ，路径变量更贴近现有 <code>`SdModelQueryController`</code> 风格。 |
| 4. None of the above | Optionally, add details in notes (tab). |

tab to add notes | enter to submit answer | </> to navigate questions | esc to interrupt

7.最佳实践与落地建议



3.展现详细的执行计划

• Proposed Plan

SdModelExtController 新增模型输出键字段接口

Summary

在 `SdModelExtController` 新增后台组件调用接口：根据 `modelId` 和 `version` 查询指定模型输出参数，仅返回 `ModelOutput` 中的 `paramKey` 与 `field`。接口使用现有 `ModelQueryManager#getModelOutputs(modelId, version, keyword)` 读取模型输出，避免重复解析数据库 `outputs` 字段。

Key Changes

- 新增接口：GET `/rs/model/ext/outputs/keys`
 - 入参：`modelId`、`version`，均使用 `@RequestParam`
 - 出参：`BsfResponse<List<ModelOutputKeyFieldVO>>`
 - `version` 传 `null` 或 `0` 时沿用现有 `ModelQueryManager` 逻辑查询最新版本
- 新增 VO：`ModelOutputKeyFieldVO`
 - 字段：`paramKey`
 - 字段：`field`
 - 放在模型 VO 包下，使用 Lombok `@Data`，不新增 Swagger 注解
- 控制器实现：
 - 注入复用已有 `modelQueryManager`
 - 调用 `modelQueryManager.getModelOutputs(modelId, version, null)`
 - 将 `ModelOutput` 列表映射为只包含 `paramKey`、`field` 的 VO 列表
 - Controller 新增方法添加完整 Javadoc，描述接口功能、参数和返回值
 - 不新增 `@ApiOperation`、`@ApiParam` 等 Swagger 相关代码

Test Plan

7.最佳实践与落地建议



3.展现详细的执行计划

```
## Test Plan

- 更新 SdModelExtControllerTest
  - 新增测试：传入 modelId=m1、version=2 时调用 modelQueryManager.getModelOutputs("m1", 2, null)
  - 验证响应非空，返回数据中只承载 paramKey 和 field
  - 覆盖 field 为空时仍透传为空，不做额外兜底
- 运行目标测试：
  - mvn -pl sdm-model/sdm-model-service -Dtest=SdModelExtControllerTest test

## Assumptions

- 接口路径采用推荐方案：GET /rs/model/ext/outputs/keys?modelId=...&version=...
- 返回类型采用新增 VO，不返回完整 ModelOutput
- 本次只新增后台组件读取接口，不调整现有 /enable/models/outputs 和 SdModelQueryController 的接口行为

- Worked for 9m 19s

Implement this plan?
> 1. Yes, implement this plan      Switch to Default and start coding.
  2. Yes, clear context and implement Fresh thread. Context: 19% used.
  3. No, stay in Plan mode         Continue planning with the model.

Press enter to confirm or esc to go back
```

7.最佳实践与落地建议



4.运行过程实时展示计划执行情况

- Updated Plan

- ✓ 新增只包含 paramKey 和 field 的输出 VO
- ✓ 在 SdModelExtController 增加后台组件查询接口
- ✓ 补充控制器单元测试并运行目标测试

5.运行结束，汇总执行情况

- 已实现新增接口：GET /rs/model/ext/outputs/keys?modelId=...&version=..., 返回 BsfiteResponse<List<ModelOutputKeyFieldVO>>, 只包含 paramKey 和 field。实现复用了现有 modelQueryManager.getModelOutputs(modelId, version, null), 未重复解析模型输出。

改动位置：

- sdm-model/sdm-model-service/src/main/java/cn/com/bsfit/sd/model/controller/SdModelExtController.java:206
- sdm-model/sdm-model-pojo/src/main/java/cn/com/bsfit/sd/model/vo/ModelOutputKeyFieldVO.java:1
- sdm-model/sdm-model-service/src/test/java/cn/com/bsfit/sd/model/controller/SdModelExtControllerTest.java:143

验证已通过：

```
mvn -pl sdm-model/sdm-model-service -am "-Dtest=SdModelExtControllerTest"
"-Dsurefire.failIfNoSpecifiedTests=false" test
```

测试结果：Tests run: 15, Failures: 0, Errors: 0, Skipped: 0。

7.最佳实践与落地建议

AGENTS.md 使用建议

当某种提示模式有效后，下一步是停止手动重复它。这就是 AGENTS.md 的作用。

一个好的 AGENTS.md 应包括：

- 仓库结构和重要目录
- 如何运行项目构建、测试和 lint 命令
- 工程规范和 PR 期望
- 约束和禁止规则完成标准以及验证方式

如果文件变得过大，应保持主文件简洁，并引用其他 markdown 文件（如规划、代码评审或架构文档）

分享：

- `./codex/AGENTS.md`
- 项目根目录下的AGENTS.md

7.最佳实践与落地建议



邦盛科技
Bangsun Technology

Config.toml 配置建议

```
model = "gpt-5.5"
model_reasoning_effort = "medium"
project_doc_fallback_filenames = ["CLAUDE.md"]
project_doc_max_bytes = 131072
approvals_reviewer = "auto_review"
approval_policy = "on-request"
sandbox_mode = "workspace-write"
[sandbox_workspace_write]
network_access = true
writable_roots = [
    'D:\maven\repo-maven-3.8.9',
    'D:\IdeaProjects'
]
[projects.'\\?\D:\IdeaProjects\sdm-v7.36.x-标准版\sdm-parent']
trust_level = "trusted"
```

Codex启动时的默认模型选择

额外的用户指令文档配置，这里是兼容CLAUDE.md文档

最大能载入到提示中的用户指令文档大小，超过部分截断

建议配置的安全权限，在保证安全的同时，减少申请

Windows系统特有配置，建议这样配置，减少申请

Cli 底部状态展示条配置

在SdModelExtController类中新增一个接口，该接口给后台组件提供服务，入参为模型id和版本，出参为ModelOutput中的paramKey和field

gpt-5.5 medium · Context 63% left · 5h 93% · weekly 83% · Fast off

Plan mode (shift+tab to cycle)

MCP配置，建议不常用的服务enable设置为

false，减少提示长度和本地资源损耗

```
[mcp_servers.openaiDeveloperDocs]
url = "https://developers.openai.com/mcp"
```

7.最佳实践与落地建议

常见问题与解决方案:

1.自动审核超时:

解决方法: VPN开启TUN(虚拟网卡)代理模式

```
⚠Automatic approval review approved (risk: low, authorization: unknown): Auto-review returned a low-risk allow decision.  
✓Auto-reviewer approved codex to run $path = "C:\Users\bangsun\Desktop\公司\2026\技术分享\Codex初探\Codex初探.md" ... this time
```

2.第一轮对话总是失败连接5轮后才行:

```
> 我有哪些 skills  
  
◦ Reconnecting... 2/5 (40s • esc to interrupt)  
  ↳ Timeout waiting for child process to exit
```

解决方法: VPN开启TUN(虚拟网卡)代理模式

7.最佳实践与落地建议

常用指令：

1. **/model**: 切换模型，选择推理强度
2. **/fast**: 开启或关闭**fast**模式
3. **/new**: 清空上下文开启新的会话（建议一个任务一个会话）
4. **/resume**: 选择并继续历史会话
5. **/plan**: 计划模式执行
6. **/status**: 查看**codex**控制面板（5小时、一个月额度剩余，加载的**Agents.md**文件等
7. **/statusline**: 配置底部**bar**显示内容
8. **/fork**: 开一个带有当前会话上下文信息的新会话
9. **/theme**: 选择**cli**中展示代码的风格

7.最佳实践与落地建议

常用指令：

1. **/model**: 切换模型，选择推理强度
2. **/fast**: 开启或关闭**fast**模式
3. **/new**: 清空上下文开启新的会话（建议一个任务一个会话）
4. **/resume**: 选择并继续历史会话
5. **/plan**: 计划模式执行
6. **/status**: 查看**codex**控制面板（5小时、一个月额度剩余，加载的**Agents.md**文件等
7. **/statusline**: 配置底部**bar**显示内容
8. **/fork**: 开一个带有当前会话上下文信息的新会话
9. **/theme**: 选择**cli**中展示代码的风格



8. ChatGPT 会员订阅最佳实践

8. ChatGPT 会员订阅最佳实践

订阅方式:

1. 代充（正规的代充价格往往高于官方价格很多，不正规的容易被封）
2. 使用国外银行卡官网订阅（国内办理的**visa**卡也不行，门槛较高）
3. 使用**apple store/google play**订阅（正规、门槛较低、价格便宜）

8. ChatGPT 会员订阅最佳实践

Apple store低价订阅方法（80¥）：我全程开了新加坡节点，据说不开也行

1. 官网登录创建土区苹果账号：[创建你的 Apple 账户](#)

创建你的 Apple 账户

只需一个 Apple 账户，即可访问 Apple 所有内容。
已有 Apple 账户？[登录](#)

名字

姓氏

国家或地区
土耳其

▼

出生日期 ?

年 ▼

月 ▼

日 ▼

name@example.com

密码

确认密码

国家/地区选项
+86 (中国大陆)

▼

电话号码

?

<https://www.meiguodizhi.com/tr-address>

土耳其地址信息生成器
(网站) 生成的土耳其人
名

选择土耳其

最好使用163邮箱（gmail邮箱我没注册成功）

使用中国大陆手机号就行
如果注册不成功更换手机号

8. ChatGPT 会员订阅最佳实践

Apple store低价订阅方法:

2. 更改**apple store**地区:

最好在**iphone**或**ipad**上操作, **macbook**有概率不行

2.1 将apple store中的账号退出

2.2 点击该连接跳转至apple store, 登录新注册的土耳其账号

<itms-apps://itunes.apple.com/WebObjects/MZStore.woa/wa/resetAndRedirect?dsf=143480&cc=tr>

<https://www.meiguodizhi.com/tr-address>

2.3 下载一款外网app固定地区 (不然后续地区大概率会回退)

2.4 下载chatggpt app

2.5 购买苹果礼品卡 (最好到正规、大型的平台购买, 不然容易被封禁)

<https://oyunfor.com/> 最正规, 但需要国内visa卡支付

<https://seagm.com/> 相对正规, 支持支付宝支付

2.6 兑换礼品卡, 在chatgpt app内使用礼品卡订阅 (chatgpt内需要使用除香港外的其它地区节点)

1. <https://openai.com/index/unrolling-the-codex-agent-loop/>
2. https://developers.openai.com/api/docs/guides/compaction?utm_source=chatgpt.com
3. <https://developers.openai.com/codex/learn/best-practices>
4. <https://developers.openai.com/codex/concepts/sandboxing>
5. <https://developers.openai.com/api/docs/guides/tools-web-search>
6. <https://developers.openai.com/codex/agent-approvals-security>
7. <https://developers.openai.com/codex/config-basic>
8. <https://context7.com/docs/resources/all-clients#openai-codex>
9. <https://developers.openai.com/codex/mcp>
10. <https://developers.openai.com/codex/skills>
11. <https://github.com/openai/skills>
12. <https://developers.openai.com/codex/subagents>